

3643.垂直翻转子矩阵

题目描述 通过 题解 提交记录

全部提交记录

通过 674 / 674 个通过的测试用例

白日梦家 提交于 2026.03.11 20:08

百度面试突击手册

众里寻你百度家

执行用时分布

0 ms 击败 100.00%

消耗内存分布

33.23 MB 击败 34.67%

代码 C++

```
1 class Solution {
2 public:
3     vector<vector<int>>> reverseSubmatrix(vector<vector<int>>& grid, int x, int y, int k) {
4         int top = x;
5         int bottom = x + k - 1;
6         int startCol = y;
7         int endCol = y + k;
8
9         while (top < bottom) {
10             for (int col = startCol; col < endCol; col++) {
11                 swap(grid[top][col], grid[bottom][col]);
12             }
13             top++;
14             bottom--;
15         }
16         return grid;
17     }
18 };
```

行 13, 列 19 已存储

测试用例 测试结果

```
1  class solution { //整体思想，先固定两个行指针，交换列元素，走到列尾后，两个行指针向中间行
    2      public:
    3          vector<vector<int>>> reverseSubmatrix(vector<vector<int>>& grid, int x,
    4              int y, int k) {
    5              int top = x; //头部行指针初始化
    6              int bottom = x + k - 1; //尾部行指针初始化
    7              int startCol = y; //初始列标
    8              int endCol = y + k; //列尾坐标
    9
    10             while (top < bottom) {
    11                 for (int col = startCol; col < endCol; col++) {
    12                     swap(grid[top][col], grid[bottom][col]);
    13                 } //列元素交换完毕
    14                 top++;
    15                 bottom--; //两个行指针向中间行进行一步靠拢
    16             }
    17             return grid;
    18         }
    19     };
```

2090.半径为k的子数组平均值

题目描述 通过 题解 提交记录

全部提交记录

通过 40 / 40 个通过的测试用例
白日梦家 提交于 2026.03.11 21:17

执行用时分布
8 ms 击败 38.58%

消耗内存分布
131.32 MB 击败 44.70%



代码 C++

```
1 class Solution {
2 public:
3     vector<int> getAverages(vector<int>& nums, int k) {
4         int size = nums.size();
5         vector<int> res(size, -1);
6         if (2 * k + 1 > size) {
7             return res;
8         }
9     }
10 }
11
12 // 查看更多
```

添加备注，例如「暴力解法」、「方法一」等

C++ 智能模式

```
1 class Solution {
2 public:
3     vector<int> getAverages(vector<int>& nums, int k) {
4         int size = nums.size();
5         vector<int> res(size, -1);
6         if (2 * k + 1 > size) {
7             return res;
8         }
9
10        int windowSize = 2 * k + 1;
11        long long sum = 0;
12        int avg = 0;
13        for (int i = 0; i < windowSize; i++) {
14            sum += nums[i];
15        }
16        avg = sum / windowSize;
17        res[k] = avg;
18
19        for (int i = k + 1; i < size - k; i++) {
20            sum = sum + nums[i + k] - nums[i - k - 1];
21            avg = sum / windowSize;
22            res[i] = avg;
23        }
24
25        return res;
26    }
27 };
```

行 19, 列 49 · 已存储 运行 提交

```
1 class Solution { //滑动窗口思想
2 public:
3     vector<int> getAverages(vector<int>& nums, int k) {
4         int size = nums.size();
5         vector<int> res(size, -1); //根据题意，将返回数组默认值都初始化为-1
6         if (2 * k + 1 > size) { //特殊情况：现有数组无法形成半径为k的子数组
7             return res;
8         }
9
10        int windowSize = 2 * k + 1; //窗口大小
11        long long windowSum = 0;
12        int avg = 0;
13        for (int i = 0; i < windowSize; i++) { //第一个符合题意的窗口
14            windowSum += nums[i];
15        }
16        avg = sum / windowSize;
17        res[k] = avg; //res[k]是以下标k为中心的半径为k的子数组的平均值
18
19        for (int i = k + 1; i < size - k; i++) { //窗口向右滑动
20            sum = sum + nums[i + k] - nums[i - k - 1];
21            avg = sum / windowSize;
22            res[i] = avg;
23        }
24
25        return res;
26    }
27 };
```